

뉴런데브클래스

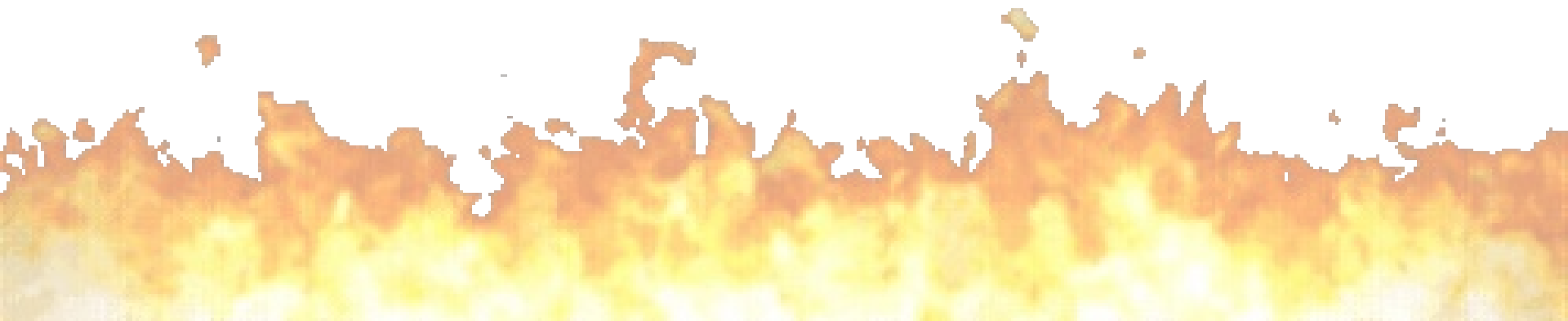
현대 AI 의 원리와 구조

#4-1 벡터디비 실습



목차

- 1 실습 환경 구성
- 2 데이터 적재
- 3 ts_rank 실습
- 4 BM25 직접 구현
- 5 종합 비교



실습 개요

1. 키워드 검색 · BM25 를 PG 안에서 직접 구현
 - 한국어 형태소 분석 → ts_rank 계열 → BM25 → 벡터 · 하이브리드
 - 강의에서 다룬 공식과 PG 동작을 실제 데이터로 확인하고 숫자로 비교
2. 준비물은 Docker Desktop 하나
 - Python · PostgreSQL 설치 불필요, 컴포즈 한 번으로 전체 환경 기동
 - Intel/AMD · Apple Silicon · Windows ARM 모두 동작
3. 모든 코드는 GitHub 저장소를 참조
 - 장표 하단의 파일 링크 → 해당 SQL · 설정 파일
 - SQL 파일 안의 '관찰:' 주석과 실제 결과를 비교하며 진행
4. 정답이 있는 한국어 데이터셋으로 검색 품질을 측정
 - KLUE-MRC: 질문마다 정답 지문 → Hit@10 · MRR

시간	단계	하는 일	기다리는 시간
00-10	① 환경 기동	compose up (빌드 동안 Dockerfile 설명)	빌드 1.5~5 분
10-13	② pgAdmin	localhost:5050 → Query Tool	-
13-16	③ 데이터 적재	loader 실행 → 텍스트 적재 완료	1~2 분 (임베딩은 뒤에서)
16-33	④ ts_rank	sql/10 → 14	14 번 평가 약 2 분
33-52	⑤ BM25 구현	sql/20 → 24	21 번 구축 약 30 초
52-60	⑥ 종합 비교	sql/25	약 30 초

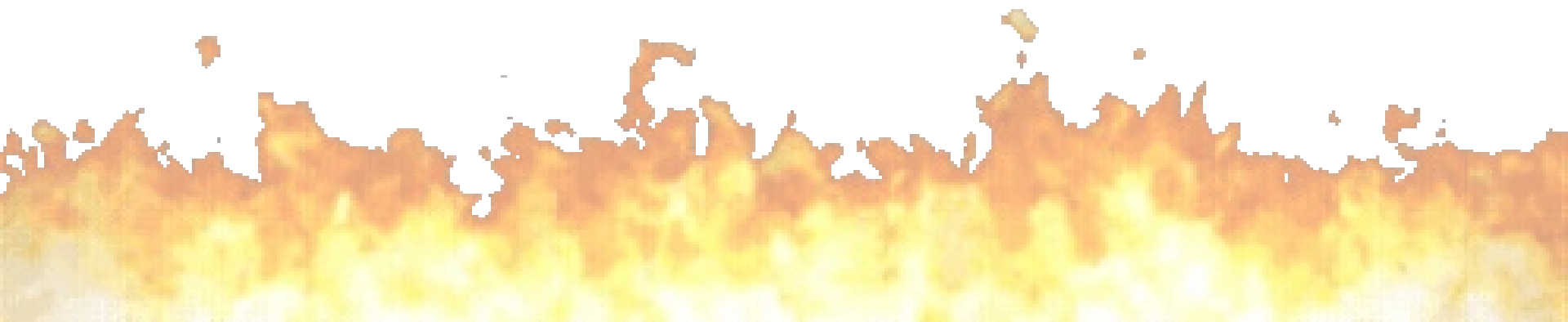
실측 (캐시 없는 최초 실행 , 12 코어 · 8GB): compose up 1 분 31 초 → 텍스트 적재 완료 2 분 33 초 → 임베딩 완료 5 분 23 초

SQL 실행 합계 약 3 분 30 초 . 4 코어 · 느린 회선에서 2~3 배가 걸려도 임베딩은 ④⑤ 동안 끝나 60 분 안에 들어온다 .

경로 (4 강실습 /)	역할
docker-compose.yml	db · pgadmin · loader 3 개 서비스
db/Dockerfile	pgvector(PG17) + mecab-ko + mecab-ko-dic + textsearch_ko
db/init/*.sql	최초 1 회 : 익스텐션 · 스키마 (docs, questions)
pgadmin/	서버 자동 등록 (servers.json) · 비밀번호 파일 (pgpass)
loader/load.py	KLUE-MRC 다운로드 → COPY → 임베딩 → HNSW
sql/10 ~ 14	ts_rank 실습 · 품질 측정
sql/20 ~ 25	BM25 구현 · 트리거 · 종합 비교
slides/	이 장표

git clone https://github.com/hikaMaeng/-ai-.git → cd -ai-/4 강실습

실습 환경 구성



도커 컴포즈 한 장으로

1. 서비스 3 개

- db : pgvector(PG17) + 한국어 형태소 분석기 , 포트 5432
- pgadmin : 웹 대시보드 , 포트 5050
- loader : 데이터 적재 도구 (profiles: tools → up 으로는 안 뜨고 run 으로만)

2. 실행

```
docker compose up -d --build  
docker compose exec db psql -U lab -d lab
```

- 첫 빌드만 3~5 분 , 이후 몇 초

3. 볼륨

- pgdata · pgadmin · hfcache : 재실행해도 다시 받지 않음
- . /sql 을 db(/sql) 와 pgAdmin 파일 브라우저 양쪽에 마운트

4. 초기화 스크립트

- db/init 은 최초 1 회 : vector · textsearch_ko 익스텐션 , docs · questions 스키마
- 처음부터 다시 : docker compose down -v

1.pgvector 공식 이미지 위에 3 개를 소스 빌드

- mecab-ko : 형태소 분석 엔진
- mecab-ko-dic : 세종 말뭉치 기반 사전
- textsearch_ko : mecab 을 PG 전문검색 파서 · 사전 (korean) 으로 연결

2. 멀티스테이지 빌드

- builder 에서 컴파일 , 최종 이미지에는 산출물 (.so · 사전) 만 복사

3.ARM 합정

- 소스 동봉 config.guess(2014 년판) 가 aarch64 를 모름
- autotools-dev 의 최신 config.guess/sub 로 덮어써서 해결

4. 확인

```
SELECT to_tsvector('korean', ' 무궁화꽃이 피었습니다 ');  
→ ' 꽃 ':2 ' 무궁화 ':1 ' 피 ':3
```

pgAdmin 자동 접속

1. 로그인 없는 데스크톱 모드

- PGADMIN_CONFIG_SERVER_MODE=False
- PGADMIN_CONFIG_MASTER_PASSWORD_REQUIRED=False

2. 서버 자동 등록

- servers.json → 4 강실습 그룹 · lab (pgvector-ko)

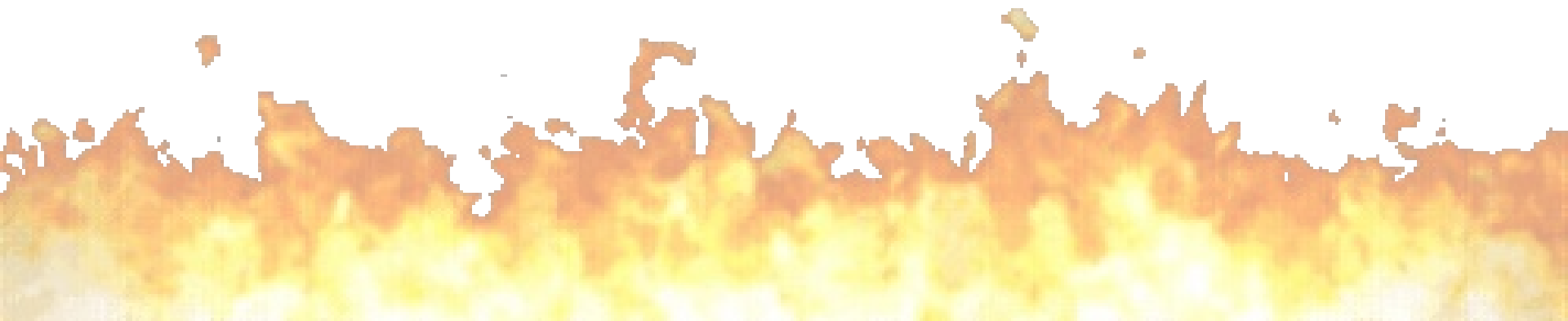
3. pgpass 권한 함정

- libpq 는 권한이 0600 이 아닌 pgpass 를 무시
- Windows 에서 마운트한 파일은 0777 로 보임 → entrypoint 에서 복사 후 chmod 600

4. 사용법

- http://localhost:5050 → lab (pgvector-ko) → Query Tool
- 폴더 아이콘 (Open File) → sql/ → 파일 선택
- 블록을 드래그하고 F5 (선택 영역만 실행)

데이터 적재



KLUE-MRC 데이터셋

1. 허깅페이스 `klue/klue · mrc · validation` (CC BY-SA 4.0)

- 질문 5,841 개 → 지문 중복 제거 후 5,309 개
- 출처 : 한국경제 2,982 · 위키백과 1,623 · 아크로팬 704

2. 고른 이유 : 질문마다 정답 지문이 있음

- '정답 지문을 상위 10 위 안에 찾았나' (Hit@10) 로 방법 간 비교 가능

3. datasets 라이브러리 없이 parquet 직접 다운로드

- 허깅페이스가 자동 변환해 둔 `/api/datasets/klue/klue/parquet/mrc/validation`

4. 지문 규모

- 평균 약 1,000 자 , 문서당 고유 형태소 평균 171 개 (31~358)

로더의 2 단계 적재

1. 텍스트 적재 (약 1 분)

```
docker compose run --rm loader
```

-parquet → md5 로 지문 중복 제거 → COPY

-tsv 는 생성 컬럼 : COPY 하는 동안 형태소 분석까지 끝남

2. '✓ 텍스트 적재 완료' 가 뜨면 창은 두고 바로 ts_rank 실습으로

3. 임베딩 (뒤에서 4~10 분 , 25 번에만 필요)

-paraphrase-multilingual-MiniLM-L12-v2 · 384 차원 · 입력 128 토큰

-UPDATE 로 채운 뒤 HNSW(vector_cosine_ops) 인덱스

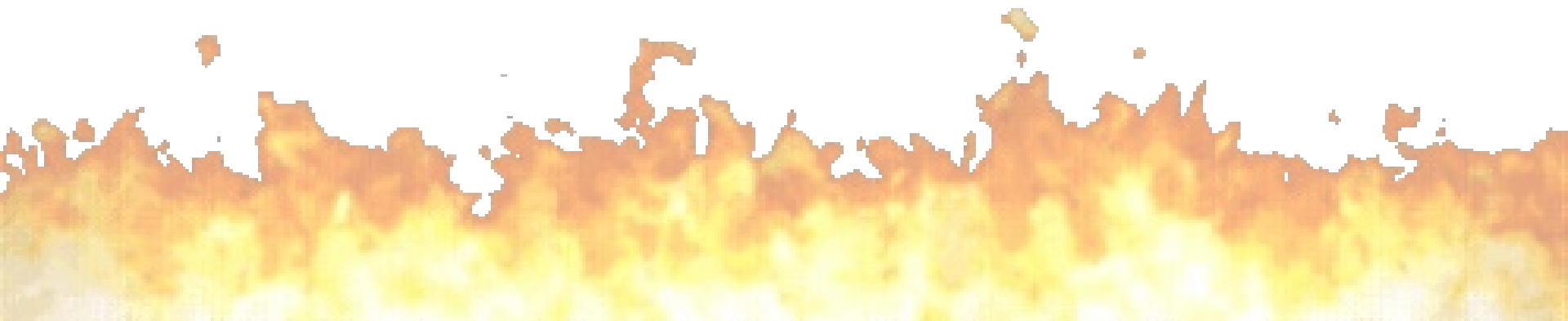
4. 스키마

-docs(title, content, tsv, embedding)

-tsv = setweight(제목 , 'A') || setweight(본문 , 'D') ← 강의 슬라이드 예제 그대로

-questions(question, answer, doc_id → 정답 지문)

ts_rank 실습



형태소 분석과 tsvector

1.simple vs korean

- simple : ' 고양이가 ':1 ' 사료를 ':2 ... 조사가 붙은 채 → 서로 못 찾음
- korean : ' 고양이 ':1,4 ' 사료 ':2,5 ' 먹 ':3 ' 추천 ':6

2.mecabko_analyze 로 품사 확인

- 가셨습니다 → 갈 /VV 시 /EP 었 /EP 습니다 /EF
- 인공지능모델학습데이터 → 인공지능 · 모델 · 학습 · 데이터

3.ts_debug : 토큰별 처리 사전

- 한글 → korean_stem, 영문 → english_stem, 숫자 → simple

4.tsvector = 형태소 → 위치 목록 + 가중치 (A~D)

- GIN 인덱스로 @@ 매칭 0.4ms

ts_rank - 가중치와 포화

1. 구역 가중치 $\{D, C, B, A\} = \{0.1, 0.2, 0.4, 1.0\}$

- 제목 (A) 에 나온 문서가 본문 (D) 에만 나온 문서보다 위로

2. 같은 단어를 n 번 반복한 문서

-ts_rank : $0.061 \rightarrow 0.076 \rightarrow 0.083 \dots 0.094$ (포화)

-ts_rank_cd : $0.1 \rightarrow 0.2 \rightarrow 0.3 \dots 1.0$ (선형)

-PG 소스 : j 번째 등장마다 $w/(j+1)^2$ 누적 $\rightarrow \pi^2/6$ 로 수렴

3.AND 의 함정

- '말라카이트에서 나온 색깔을 사용한 에디션은 ?' $\rightarrow$ 형태소 5 개 AND $\rightarrow$ 0 건

4.OR 로 풀면 정답은 2 위

-1 위는 제목에 '에디션' 이 들어간 다른 기사

- 코퍼스에 1 건뿐인 '말라카이트' 의 희귀함을 ts_rank 는 모름 $\rightarrow$ IDF

ts_rank_cd - 커버 밀집도

1. AND 질의 : 질의어가 많수록 감점

- 붙어 있음 $0.1 \cdot 8$ 단어 사이 $0.011 \cdot 30$ 단어 사이 0.003

2. 커버가 여러 번이면 합산

- 1 개 0.1 · 2 개 0.225 · 3 개 0.4

3. OR 질의에서는 밀집도 효과가 사라짐

- 단어 하나만으로 커버 성립 → 거리와 무관하게 같은 점수

- 문장형 질문을 OR 로 풀면 cd 의 장점이 약해진다

4. 위치 정보 필수

- strip() 으로 위치를 지운 tsvector 는 0 점

길이 정규화 옵션

1. ts_rank 세 번째 인자

- 0 없음 $\cdot 1 \div (1 + \log \text{길이}) \cdot 2 \div \text{길이} \cdot 8 \div \text{고유단어수} \cdot 32 \text{ rank} / (\text{rank} + 1)$
- 여러 옵션을 | 로 조합 (예 : 1|32 = 로그 감쇄 후 0~1 로)

2. opt1 은 긴 문서를 살짝 , opt2 는 크게 감점

- ' 코로나 백신 접종 ' 상위 10 개 평균 길이 : opt0 1,279 자 $\rightarrow$ opt1·opt2 825 자

3. 한계 : 코퍼스 평균 길이 (avgdl) 를 모름

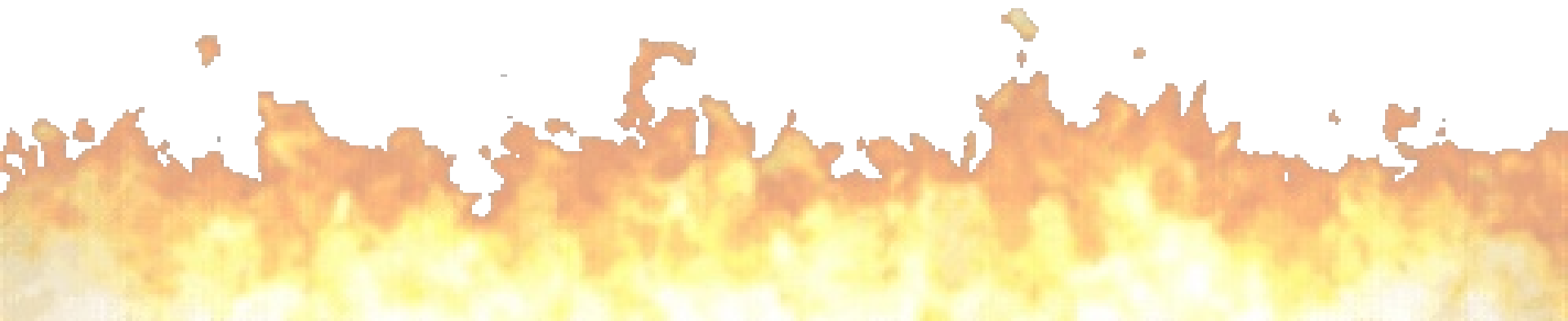
- 길이는 그 문서 하나의 값일 뿐
- ' 평균보다 긴가 짧은가 ' 로 보간하는 BM25 의 b 를 흉내 낼 수 없다

방법	Hit@10	MRR	해석
AND (plainto_tsquery)	0.048	0.044	정확하지만 거의 못 찾음 (재현율 붕괴)
OR + ts_rank	0.484	0.354	흔한 단어가 많은 문서에 끌려감
OR + ts_rank_cd	0.524	0.329	Hit 는 약간 높고 순위는 낮음
OR + ts_rank 정규화 1 (선택)	0.484	0.354	정답 순위 198 건 변동 , 합계는 거의 동일
OR + ts_rank 정규화 2 (선택)	0.374	0.233	짧은 지문을 과하게 끌어올림

질문 500 개 샘플 ($id \% 11 = 0$). MRR = 정답 순위 역수의 평균 (1 위 1.0, 2 위 0.5, 10 위 밖 0). 결과는 eval_result 에 저장

지문 길이가 고른 코퍼스라 로그 정규화 (1) 의 효과가 작다 → 긴 청크 RAG 에는 opt1, 극단적으로 짧은 청크에만 opt2

BM25 직접 구현



공식을 테이블로

1. BM25 공식

- $BM25(D, Q) = \sum IDF(t) \cdot tf \cdot (k1 + 1) / (tf + k1 \cdot (1 - b + b \cdot |D| / avgdl))$

- $IDF(t) = \ln(1 + (N - df + 0.5) / (df + 0.5))$ ← Lucene 변형

2. 필요한 값 → 테이블

- $tf \rightarrow bm25_tf(term, doc_id, tf)$: 역색인, PK(term, doc_id)

- $|D| \rightarrow bm25_doclen \cdot N$, $avgdl \rightarrow bm25_stats$ (항상 1행)

- $df \rightarrow bm25_df \cdot IDF \rightarrow bm25_idf$ (df 와 N 으로 계산한 캐시)

3. 자료는 이미 tsvector 안에 있다

- $unnest(tsv) \rightarrow lexeme = term$, $cardinality(positions) = tf$

- 형태소 분석은 끝났으니 '세고 점수 매기기'만 구현

- 주의 : tsvector 는 형태소당 위치를 256 개까지만 저장

일괄 구축과 IDF

1. 집합 연산 5 번이면 끝 (약 15 초)

- 역색인 908,436 행 · 고유 단어 64,877 개 · N 5,309 · avgdl 281.3

2. 흔한 단어는 $IDF \approx 0$

- 것 0.23 · 있 0.26 · 들 0.28 · 등 0.30 (슬라이드 '은 · 는 · 이 · 가 거의 0 점 ')

3. 희귀 단어는 고점

- 말라카이트 8.17 · 트랜스포머 7.66 · 에디션 5.77 · 반도체 4.31

4. $df = 1$ 인 단어가 31,860 개 (49%)

- 대부분의 단어는 한두 문서에만 나온다 (지프의 법칙)

5. IDF 는 N 이 바뀌면 전부 바뀐다

-bm25_refresh_idf() 함수로 분리해 두고 필요할 때 전체 재계산

점수 분해와 검색 함수

1. 정답 지문 점수 $24.3 = \text{말라카이트 } 12.96 + \text{에디션 } 11.38$

- 말라카이트 : tf 3, IDF 8.2 / 에디션 : tf 10 이지만 포화

2. k1 이 클수록 포화가 늦다

- tf=10 일 때 tf 항 : $k1 \ 1.2 \rightarrow 1.96 \cdot 2.0 \rightarrow 2.50 \cdot 5.0 \rightarrow 4.00$ (상한 $k1+1$)

3. b : 평균보다 긴 문서 감점 , 짧은 문서 가점

- $|D|/avgdl = 4$ 일 때 : $b \ 0 \rightarrow 1.57 \cdot 0.75 \rightarrow 0.96 \cdot 1 \rightarrow 0.85$

4. bm25_search(q, k, k1, b)

- 질의도 to_tsvector('korean') → 역색인 PK 로 질의어 행만 읽음 (약 35ms)

- ts_rank 에서 2 위였던 정답이 1 위로

트리거로 동적 갱신

1. 문장 단위 트리거 + 전이 테이블

- REFERENCING NEW TABLE AS new_rows → 1 만 건 INSERT 도 1 회 실행
- 전이 테이블은 이벤트 하나짜리 트리거만 → INSERT · UPDATE · DELETE 3 개

2.UPDATE = 옛 문서 빼기 + 새 문서 더하기

- tsv 가 그대로인 UPDATE(임베딩만 수정) 는 건너뛴

3.IDF 는 건드린 단어만 즉시 갱신

- 나머지는 옛 N 기준으로 남음 → 주기적으로 bm25_refresh_idf()
- Lucene 의 세그먼트 병합과 같은 발상

4.bm25_rebuild()

- 트리거가 꺼져 있던 동안의 불일치 복구용 전체 재구축

트리거 검증

1. INSERT

- 새 단어 '큐비트' 즉시 검색 · 말라카이트 df 1→2, IDF 8.17→7.66

2. UPDATE

- 본문에서 '큐비트' 제거 → 사전에서도 삭제
- source 만 바꾼 UPDATE → 통계 변화 없음

3. DELETE

- N · total_len 원래 값으로 복귀

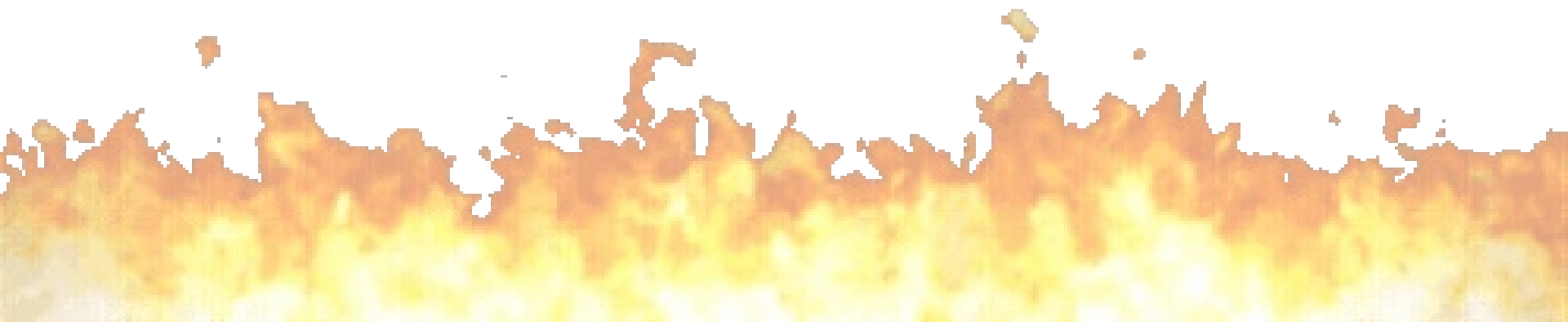
4. 정합성 대조 : 트리거 유지값 vs 처음부터 재계산

- tf · df · N 불일치 0 건
- IDF 오차 7 건 = 설계상 허용 → bm25_refresh_idf() 후 0 건

5. 로더 재실행도 트리거가 처리

- DELETE 5,309 행 → COPY 5,309 행 = 트리거 2 회

종합 비교



방법	Hit@10	MRR	비고
OR + ts_rank	0.484	0.354	PG 기본
OR + ts_rank_cd	0.524	0.329	밀집도
BM25 (직접 구현)	0.938	0.838	IDF + avgdl 길이 보간
벡터 (MiniLM, 지문 통째)	0.376	0.204	HNSW 코사인
하이브리드 RRF	0.896	0.523	$\sum 1/(60 + \text{rank})$

BM25 파라미터 : $k1\ 0.5 \rightarrow 0.948 \cdot b\ 0 \rightarrow 0.948 \cdot$ 기본 $(1.2, 0.75) \rightarrow 0.938 \cdot k1\ 2.0 \rightarrow 0.930$

BM25 만 맞힌 질문 : 고유명사 · 숫자 (슈트레제만 , RIR 중위값 , 코페르니쿠스 ...)

벡터만 맞힌 질문 : 바꿔 말하기 (' 실존하는 발광체로 만들 수 있는 색깔 ' ...)

결과 해석과 과제

1. BM25 가 `ts_rank` 의 두 배

- 차이는 IDF 와 평균 길이 기반 길이 보간뿐 — 같은 `tsvector` 를 썼다

2. 데이터의 성격

- KLUE-MRC 질문은 지문의 고유명사 · 숫자를 그대로 사용 → 키워드 검색에 유리
- 지문 길이가 고름 → $b=0$ 이 오히려 좋다

3. 벡터가 약한 이유 = 청킹

- 입력 128 토큰 → 지문 앞 2~3 문장만 벡터에 담김
- 정답이 뒤쪽에 있으면 못 찾음 (강의 : ' 청킹이 가장 중요 '

4. RRF 는 동등 가중

- 약한 쪽이 강한 쪽의 순위를 끌어내림

5. 과제

- 지문을 문장 단위로 청킹해 다시 임베딩 → 25 번 재측정
- 가중 RRF (BM25 쪽 가중 $\uparrow$) · 질의어 빈도 (`qtf`) 반영 · BM25F (제목 / 본문 따로)

정리와 문제 해결

1. 끝나면

```
docker compose down          # 컨테이너만 정지 ( 데이터 유지 )  
docker compose down -v      # 데이터까지 삭제 , 처음부터
```

2. 포트 충돌 (5432 / 5050 사용 중)

```
DB_PORT=15432 PGADMIN_PORT=15050 docker compose up -d
```

3.pgAdmin 이 안 열림

- 첫 기동은 20~30 초 · docker compose logs pgadmin

4.25 번 벡터 결과가 0

- 임베딩 진행 중 → loader 창의 '완료' 확인 또는 --embed-only

5. 주의

- 로더를 docker compose run --build 로 실행하면 db 가 재생성되어 쿼리가 끊긴다